



Object Oriented Programming

Pass Task 11.1: Clock in Another Language

Overview

When learning a new language it is always best to create a small program that you are familiar with. In this task you will recreate the Clock class from the previous task in a new programming language.

- Purpose:** See that the principles you have learnt apply equally to other object oriented programming languages.
- Task:** Implement your Clock class and supporting Counter class in a different object oriented programming language.
- Time:** End of Week 12, Friday 25 Oct 2024, 23:59:00 (Firmed)

Submission Details

All students have access to the Adobe Acrobat tools. Please print your solution to PDF and combine it with the screenshots taken for this task.

- Program source code
- Screenshot of program execution
- Screenshots comparing the memory usage and execution time between your clock application implemented in C# and your newly selected programming language.

Instructions

Review your design for the clock from the previous pass task, and use this to implement the Clock in a different OO programming language. You can use any OO programming language except for C#, and you must recreate the design from the previous task.

Hints. See next page for references regarding how to measure memory usage and execution time with C#, Java, and Python.

Note: We do not require unit test implementation for this task. You could work out how to do unit test-ing in the other language at a later stage at your own study.

Assessment Criteria

Make sure that your task has the following in your submission:

- The program is implemented correctly based on the original clock design.
- Code must mostly follow the coding conventions of your chosen language.
- The “Universal Task Requirements” (see Canvas) have been met.

Hints.

With C#, you can add the following code at the end of your Program.cs to measure the physical memory usage of your current process.

```
//Get the current process
System.Diagnostics.Process proc =
System.Diagnostics.Process.GetCurrentProcess();
Console.WriteLine("Current process: {0}", proc.ToString());
//Display the total physical memory size allocated for the current
process Console.WriteLine("Physical memory usage: {0} bytes",
proc.WorkingSet64);
// Display peak memory statistics for the process.
Console.WriteLine("Peak physical memory usage {0} bytes",
proc.PeakWorkingSet64);
```

- Reference for the memory usage with C#: <https://learn.microsoft.com/en-us/dotnet/api/system.diagnostics.process.totalprocessortime?view=net-8.0>

To measure the elapsed execution time, you can use the Stopwatch class in .Net framework. Please refer to the following link for an example.

<https://learn.microsoft.com/en-us/dotnet/api/system.diagnostics.stopwatch?view=net-6.0>

With Java, you can use the Runtime class to measure the memory usage. The following code will help you measure the used memory in bytes.

```
// Get the total memory available to the JVM in bytes
long totalMemory = runtime.totalMemory();
// Get the free memory available to the JVM in bytes
long freeMemory = runtime.freeMemory();
// Calculate the used memory in bytes
long usedMemory = totalMemory - freeMemory;
```

- Reference for the memory usage with Java: <https://www.geeksforgeeks.org/java-runtime-totalmemory-method/>
- Reference for the Stopwatch class to measure the elapsed time in Java. <https://introcs.cs.princeton.edu/java/stdlib/Stopwatch.java.html>

With Python, you can use either Tracemalloc or Psutil library to monitor the memory usage. Reference is provided below.

<https://www.geeksforgeeks.org/monitoring-memory-usage-of-a-running-python-program/>

```

import tracemalloc
import time
class Counter:
    def __init__(self, name):
        self._name = name
        self._count = 0

    def increment(self):
        self._count += 1

    def reset(self):
        self._count = 0

    @property
    def name(self):
        return self._name

    @name.setter
    def name(self, value):
        self._name = value

    @property
    def ticks(self):
        return self._count

class Clock:
    def __init__(self, counter):
        self.seconds = Counter("seconds")
        self.minutes = Counter("minutes")
        self.hours = Counter("hours")

    def update(self):
        self.seconds.increment()
        if self.seconds.ticks == 60:
            self.seconds.reset()
            self.minutes.increment()
        if self.minutes.ticks == 60:
            self.minutes.reset()
            self.hours.increment()
        if self.hours.ticks == 24:
            self.hours.reset()
            self.minutes.reset()
            self.seconds.reset()

    def display(self):
        return f"{self.hours.ticks:02}:{self.minutes.ticks:02}:{self.seconds.ticks:02}"

def main():

```

```
seconds = "seconds"
counter = Counter(seconds)
clock = Clock(counter)
```

```
for i in range(24):
    print(clock.display())
    clock.update()
```

```
if __name__ == "__main__":
    tracemalloc.start()
    start = time.time()
    main()
    end = time.time()
    print(end - start)
    print(tracemalloc.get_traced_memory())
```

```
# stopping the library
tracemalloc.stop()
```

```
00:00:05
00:00:06
00:00:07
00:00:08
00:00:06
00:00:07
00:00:08
00:00:07
00:00:08
00:00:09
00:00:10
00:00:11
00:00:08
00:00:09
00:00:10
00:00:11
00:00:09
00:00:10
00:00:11
00:00:11
00:00:12
00:00:12
00:00:13
00:00:14
00:00:15
00:00:16
00:00:17
00:00:14
00:00:15
00:00:16
00:00:17
00:00:16
00:00:17
00:00:17
00:00:17
00:00:18
00:00:19
00:00:20
00:00:21
00:00:22
00:00:23
0.004578113555908203
(0, 2045)
```

	ID	Time	Objects (Diff)	Heap Size (Diff)	Label
	1	3.95s	3,622 (n/a)	336.94 KB (n/a)	
0.004578113555908203 (0, 2045)					